Read-Write Separation

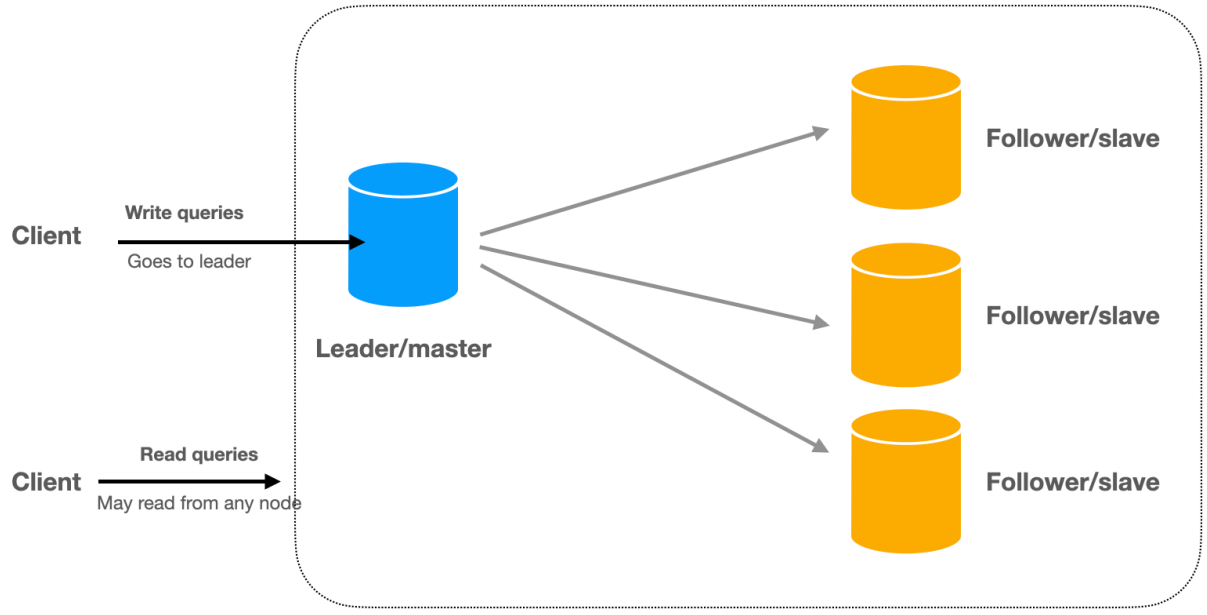
systemdesignschool.io/fundamentals/read-write-separation

A system typically have different traffic patterns for read and write operations. For example, a social media platform might have millions of users scrolling through their feeds, but only a fraction actively creating new posts. Similarly, an e-commerce platform might see far more product browsing than actual purchases, and a content platform might have many more consumers than creators. An IOT device, one the other hand, might have a very high number of writes, but a low number of reads.

This asymmetry in operation types creates an opportunity to optimize the system. We can handle read and write operations in different ways, and scale them independently.

Database Replication

The classic read-write separation pattern is database replication. In this model, a primary database takes responsibility for all write operations while multiple replica databases handle read queries. These replicas continuously synchronize with the primary, creating a distributed system that can handle significantly higher read loads.



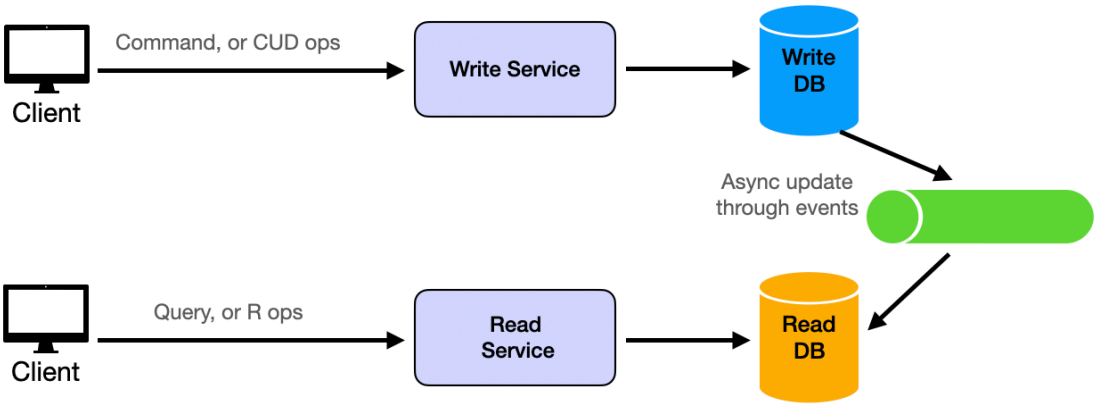
This approach brings several advantages to the table. Beyond the obvious benefit of improved read performance through load distribution, it also enhances system availability and fault tolerance. If one replica fails, others can continue serving read requests while the failed instance recovers. Additionally, you can scale your read capacity horizontally by adding more replicas as needed.

However, replication isn't without its challenges. One of the most significant issues is replication lag - the delay between when data is written to the primary and when it becomes available in the replicas. This lag can lead to temporary data inconsistencies where readers might see outdated information. Organizations must also prepare for primary database failures by implementing robust failover mechanisms.

We will cover replication in more detail in the replication section.

CQRS: Advanced Read-Write Separation

For applications with more complex requirements, Command Query Responsibility Segregation (CQRS) offers a sophisticated approach to read-write separation. Rather than simply replicating data, CQRS separates the entire data access architecture into distinct command (write, i.e. Create, Update, Delete) and query (read, i.e. Read) paths.



CRUD = Create, Read, Update, Delete

In a CQRS architecture, write operations use a normalized data model optimized for consistency and data integrity. Meanwhile, read operations work with denormalized views specifically designed for efficient querying. These two models are kept in sync through asynchronous processes, allowing each side to be optimized independently.

This separation enables remarkable flexibility in how data is stored and accessed. The write side might use a traditional relational database to ensure ACID compliance, while the read side could leverage a document database for faster queries. This flexibility makes CQRS particularly valuable for applications with complex domain models or unique performance requirements.